

GLM-5.1 via OpenRouter for Automated obsidian_rag Code Review

A copy-pasteable configuration and integration guide for reviewing `~/dev/obsidian_rag` with GLM-5.1, OpenRouter, OpenClaw, and a reasoning-enabled Python review harness. Repository: https://github.com/powskierman/obsidian_rag.git.

Prepared by Perplexity Computer

External facts used in this guide come from OpenRouter's documentation. OpenRouter's chat API uses POST <https://openrouter.ai/api/v1/chat/completions> with Authorization: Bearer <API key> authentication.¹ The OpenRouter GLM 5.1 page is published under the Z.AI provider path and describes GLM-5.1 as a coding-capable model with reasoning support.² OpenRouter's OpenClaw guide documents `~/ .openclaw/openclaw.json` as the primary OpenClaw configuration file and the model-name pattern `openrouter/<author>/<slug>`.³ OpenRouter's reasoning guide documents the reasoning request object with fields such as `enabled`, `effort`, `max_tokens`, and `exclude`.⁴

What this sets up

- A safe OpenClaw configuration for OpenRouter with GLM-5.1 as the primary model.
- A compatibility copy at `~/ .openclaw/config.json` for OpenClaw wrappers that still look for that file, while keeping `openclaw.json` canonical.
- A cascading review prompt pack focused on vector-search sensitivity, especially failure modes around broad queries such as notes.
- A Python runner that samples relevant files from `~/dev/obsidian_rag`, calls OpenRouter with reasoning enabled, and writes Markdown plus JSON review reports.

Assumptions

- Your local repo path is `~/dev/obsidian_rag`.
- Your cleaned GitHub repository is https://github.com/powskierman/obsidian_rag.git.
- The production topology uses three Macs: `canmore` is the canonical repository/codebase host, `lobster` is on the same LAN and runs OpenClaw, and `macbook` is remote over Tailscale and mirrors the repo.
- You will keep `OPENROUTER_API_KEY` in your shell, keychain, password manager, or OpenClaw auth profile. Do not commit it.
- The OpenRouter API model slug is `z-ai/glm-5.1`; the OpenClaw model string is `openrouter/z-ai/glm-5.1` because OpenClaw prefixes OpenRouter models with `openrouter/`.
- The review harness is intentionally read-only. It should run on `lobster` against a local clone or sync of `canmore`'s repo, writing reports under `reports/glm51/` without modifying application code.

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>

2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>

3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>

4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Three-Mac topology

The best operational model is to keep canmore authoritative, run OpenClaw and the OpenRouter API key on lobster, and treat macbook as a remote mirror/control surface over Tailscale. This avoids routing latency-sensitive review work through the remote macbook while still letting you trigger reviews and collect reports remotely.

Host	Role	Recommended responsibility
canmore	Canonical repo/codebase	Owns the source-of-truth ~/dev/obsidian_rag. Push cleaned changes to GitHub from here or from a trusted mirror.
lobster	OpenClaw runtime	Stores OPENROUTER_API_KEY, OpenClaw config, a local clone/sync of the repo, and GLM-5.1 review reports.
macbook	Remote Tailscale mirror	Mirrors the repo, triggers lobster jobs over SSH/Tailscale, and pulls reports back for review while offsite.

Secret placement

- Put OPENROUTER_API_KEY on lobster, because that is where OpenClaw and the Python harness should call OpenRouter.
- Keep canmore focused on source control and codebase state. It does not need OpenRouter credentials unless you also run analysis there.
- Keep macbook optional. It can trigger lobster over Tailscale, but the normal review path should not depend on macbook being online.

Integration steps

Create an OpenRouter API key on lobster: Export it as OPENROUTER_API_KEY on lobster. OpenRouter keys generally use the sk-or-... prefix in the OpenClaw integration guide.³

Install or verify OpenClaw on lobster: Run `openclaw onboard` for the wizard, or use the manual JSON below if you prefer deterministic config.³

Clone or sync canmore to lobster: Use Git-over-SSH when possible. If you need a file mirror instead, use `rsync` and exclude `virtualenvs`, `caches`, and `generated reports`.

Create the setup script on lobster's clone: Save the shell script in the next section as `~/dev/obsidian_rag/setup_glm51_openrouter_review.sh` on lobster and mark it executable.

Create the Python runner on lobster's clone: Save the Python script as `~/dev/obsidian_rag/tools/glm51_review.py` on lobster and mark it executable.

Run focused analysis on lobster: Start with `--focus vector-search --query notes`, then repeat for related weak queries. Trigger from macbook over SSH/Tailscale only as a remote-control convenience.

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Setup script

Save as `~/dev/obsidian_rag/setup_glm51_openrouter_review.sh`. It validates the repo, writes OpenClaw config, creates a compatibility `config.json`, and installs a lightweight Python environment.

```
#!/usr/bin/env bash
set -euo pipefail

REPO="${OBSIDIAN_RAG_REPO:-$HOME/dev/obsidian_rag}"
OPENCLAW_DIR="${OPENCLAW_DIR:-$HOME/.openclaw}"
OPENCLAW_CONFIG="${OPENCLAW_CONFIG:-$OPENCLAW_DIR/openclaw.json}"
COMPAT_CONFIG="${COMPAT_CONFIG:-$OPENCLAW_DIR/config.json}"
MODEL_API="${OPENROUTER_MODEL:-z-ai/glm-5.1}"
MODEL_OPENCLAW="${OPENCLAW_MODEL:-openrouter/z-ai/glm-5.1}"

if [[ ! -d "$REPO" ]]; then
    echo "Repo not found: $REPO" && 2
    exit 1
fi

if [[ -z "${OPENROUTER_API_KEY:-}" ]]; then
    echo "Set OPENROUTER_API_KEY first, e.g. export OPENROUTER_API_KEY='sk-or-...' && 2
    exit 1
fi

mkdir -p "$OPENCLAW_DIR" "$REPO/tools" "$REPO/reports/glm51"
chmod 700 "$OPENCLAW_DIR"

cat && " $OPENCLAW_CONFIG" && JSON
{
  "env": {
    "OPENROUTER_API_KEY": "\${OPENROUTER_API_KEY}"
  },
  "auth": {
    "profiles": {
      "openrouter:default": {
        "provider": "openrouter",
        "mode": "api_key"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "$MODEL_OPENCLAW",
        "fallbacks": ["openrouter/openrouter/auto"]
      },
      "models": {
        "$MODEL_OPENCLAW": {
          "reasoning": {
            "enabled": true,
            "exclude": false
          },
          "temperature": 0.15,
          "max_completion_tokens": 8192
        },
        "openrouter/openrouter/auto": {}
      }
    }
  }
}
JSON

# Some OpenClaw builds and wrappers look for config.json. Keep it in sync, but
# treat openclaw.json as canonical because that is the documented location.
cp "$OPENCLAW_CONFIG" "$COMPAT_CONFIG"
chmod 600 "$OPENCLAW_CONFIG" "$COMPAT_CONFIG"
```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

```
python3 -m venv "$REPO/.venv-glm51-review"
"$REPO/.venv-glm51-review/bin/python" -m pip install --upgrade pip requests

cat &&&EOF
Configured GLM-5.1 review workspace.
Repo:          $REPO
OpenClaw config: $OPENCLAW_CONFIG
Compat config:  $COMPAT_CONFIG
API model slug: $MODEL_API
OpenClaw model: $MODEL_OPENCLAW

Next:
  cd "$REPO"
  source .venv-glm51-review/bin/activate
  python tools/glm51_review.py --repo . --focus vector-search --query notes
EOF
```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

OpenClaw JSON configuration

OpenRouter's OpenClaw guide names `~/ .openclaw/openclaw.json` as the documented primary config file and does not mention `config.json`.³ Because you asked for `config.json`, the setup script writes the same JSON to both paths so older wrappers and documented OpenClaw can share one provider/model configuration.

```
{
  "env": {
    "OPENROUTER_API_KEY": "${OPENROUTER_API_KEY}"
  },
  "auth": {
    "profiles": {
      "openrouter:default": {
        "provider": "openrouter",
        "mode": "api_key"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/z-ai/glm-5.1",
        "fallbacks": ["openrouter/openrouter/auto"]
      },
      "models": {
        "openrouter/z-ai/glm-5.1": {
          "reasoning": {
            "enabled": true,
            "exclude": false
          },
          "temperature": 0.15,
          "max_completion_tokens": 8192
        },
        "openrouter/openrouter/auto": {}
      }
    }
  }
}
```

Why these fields

- `env.OPENROUTER_API_KEY` keeps OpenRouter available to OpenClaw without hard-coding the secret in repository files.
- `auth.profiles.openrouter:default` matches the OpenClaw auth-profile pattern documented by OpenRouter.³
- `agents.defaults.model.primary` uses `openrouter/z-ai/glm-5.1`, while the Python API uses `z-ai/glm-5.1`.
- `reasoning.enabled=true` asks OpenRouter to enable reasoning behavior; `exclude=false` asks the API to return available reasoning metadata.⁴

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>

2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>

3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>

4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Cascading prompt pack for vector-search sensitivity

Use this prompt pack either inside OpenClaw or as the conceptual cascade behind the Python runner. The point is to make GLM-5.1 review the same failure from multiple angles instead of producing a single generic code review.

```
System:
You are reviewing obsidian_rag as a senior staff engineer. Be skeptical,
test-driven, and specific. Prefer small, reversible refactors. Output JSON.

Cascade A – architecture map:
- Trace ingestion -&gt; chunking -&gt; embedding -&gt; vector store -&gt; query parser -&gt; router -&gt; response
  composer.
- Identify thresholds, top_k defaults, score normalization, metadata filters, fallback paths, and
  logging.

Cascade B – vector sensitivity:
- Check whether generic terms such as "notes" collapse to zero matches, too many matches, or
  unrelated matches.
- Inspect normalization: lowercasing, stemming/lemmatization, stopword handling, singular/plural
  handling, project slug aliases.
- Inspect vector-only search versus hybrid lexical/vector retrieval.
- Verify behavior when embeddings are missing, stale, wrong-dimension, or indexed against a
  different model.

Cascade C – refactor checks:
- Add explicit diagnostics for zero-result, low-score, and over-broad-result cases.
- Add a lexical fallback or hybrid rerank when query specificity is low.
- Separate retrieval thresholds for exact project slugs, broad concepts, and natural-language
  questions.
- Keep source lineage and do not hallucinate from empty retrieval.

Cascade D – tests:
- Add regression tests for query="notes" returning traceable results or a clear "too broad"
  diagnostic.
- Test "note" vs "notes", capitalization, empty DB, stale index, and mixed vault/SQLite retrieval.
- Log query, normalized query, candidate counts, score distribution, selected sources, and fallback
  reason.
```

Specific checks for the query “notes”

- Zero-result handling: verify that a generic but valid term does not fail silently.
- Over-broad handling: verify that a term matching many notes either reranks usefully or returns a clear broad-query diagnostic.
- Normalization: compare notes, note, Notes, and project slug aliases.
- Score thresholds: inspect whether a global threshold is too high for short queries or too low for unrelated chunks.
- Hybrid fallback: require lexical or metadata fallback when vector similarity is inconclusive.
- Lineage: require every answer to cite retrieved source chunks or return “not enough context.”

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Python reasoning-enabled analysis runner

Save as `~/dev/obsidian_rag/tools/glm51_review.py`. It calls the OpenRouter chat-completions endpoint, which supports model, messages, response format, provider routing, reasoning, temperature, max-completion-token, stream, and usage fields.¹

```
#!/usr/bin/env python3
"""
GLM-5.1 / OpenRouter code review harness for ~/dev/obsidian_rag.

Usage:
  export OPENROUTER_API_KEY="sk-or-..."
  python tools/glm51_review.py --repo ~/dev/obsidian_rag --focus vector-search --query notes
"""

from __future__ import annotations
import argparse, json, os, pathlib, subprocess, sys, textwrap, time
import requests

API_URL = "https://openrouter.ai/api/v1/chat/completions"
MODEL = os.getenv("OPENROUTER_MODEL", "z-ai/glm-5.1")
SKIP_DIRS = {".git", ".venv", ".venv-glm51-review", "__pycache__", "node_modules", ".mypy_cache",
              ".pytest_cache", "dist", "build"}
EXTS = {".py", ".md", ".toml", ".yaml", ".yml", ".json", ".sh", ".sql"}

CASCADE = [
    ("map", "Map retrieval architecture: ingestion, chunking, embeddings, vector DB, query routing, fallback search, and tests."),
    ("sensitivity", "Find sensitivity failures for broad or overloaded queries like 'notes', 'note', 'work', and project slugs."),
    ("refactor", "Suggest minimal refactors that improve query normalization, thresholds, fallback paths, observability, and regression coverage."),
    ("tests", "Produce concrete pytest or CLI regression tests, including expected behavior and failure diagnostics."),
]

def run(cmd, cwd):
    return subprocess.run(cmd, cwd=cwd, text=True, stdout=subprocess.PIPE,
                          stderr=subprocess.STDOUT).stdout.strip()

def collect_files(repo: pathlib.Path, max_files: int, max_bytes: int):
    ranked = []
    keywords = ("retriev", "vector", "embed", "chunk", "query", "search", "sqlite", "router", "rag", "test")
    for path in repo.rglob(".*"):
        if not path.is_file() or path.suffix.lower() not in EXTS:
            continue
        if any(part in SKIP_DIRS for part in path.parts):
            continue
        rel = path.relative_to(repo)
        score = sum(k in str(rel).lower() for k in keywords)
        if score or len(ranked) <= max_files // 2:
            ranked.append((score, rel))
    ranked.sort(key=lambda x: (-x[0], str(x[1])))

    chunks, total = [], 0
    for _, rel in ranked[:max_files]:
        raw = (repo / rel).read_bytes()[:max_bytes]
        text = raw.decode("utf-8", errors="replace")
        total += len(text)
        chunks.append(f"\n--- FILE: {rel} ---\n{text}")
    return "\n".join(chunks), total, [str(r[1]) for r in ranked[:max_files]]

def build_prompt(repo, focus, query, context, files, git_status, git_diff, phase_name, phase_goal):
    return f"""
You are reviewing the obsidian_rag codebase as a senior staff engineer.
Repository path: {repo}
Focus: {focus}
Regression query: {query!r}

Phase: {phase_name}
Goal: {phase_goal}

Known risk pattern:
- Vector search can become too sensitive or too brittle for generic terms.
- A query such as "notes" must not fail silently, over-filter to zero results, or return unrelated results without diagnostics.
- Prefer bounded, testable refactors over broad rewrites.

Review instructions:
1. Identify concrete risks, likely files/functions, and missing tests.
2. For each finding, include severity, evidence, why it matters, and a minimal fix.
3. Include regression tests or commands that would catch the problem.
4. Return valid JSON with keys: phase, summary, findings, tests, commands, refactor_plan.

Git status:
    
```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

```

{git_status}

Git diff, if any:
{git_diff[:12000]}

Selected files:
{json.dumps(files, indent=2)}

Code context:
{context}
"""

def call_openrouter(prompt):
    key = os.getenv("OPENROUTER_API_KEY")
    if not key:
        sys.exit("OPENROUTER_API_KEY is not set")
    headers = {
        "Authorization": f"Bearer {key}",
        "Content-Type": "application/json",
        "HTTP-Referer": "https://localhost/obsidian_rag",
        "X-Title": "obsidian_rag GLM-5.1 Review",
    }
    payload = {
        "model": MODEL,
        "messages": [
            {"role": "system", "content": "You are precise, skeptical, and test-driven. Output JSON only."},
            {"role": "user", "content": prompt},
        ],
        "reasoning": {"enabled": True, "exclude": False},
        "temperature": 0.15,
        "max_completion_tokens": 8192,
        "response_format": {"type": "json_object"},
    }
    r = requests.post(API_URL, headers=headers, json=payload, timeout=240)
    r.raise_for_status()
    data = r.json()
    choice = data["choices"][0]["message"]
    return {
        "content": choice.get("content", ""),
        "reasoning": choice.get("reasoning"),
        "reasoning_details": choice.get("reasoning_details"),
        "usage": data.get("usage"),
        "model": data.get("model", MODEL),
    }

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--repo", default=os.path.expanduser("~/dev/obsidian_rag"))
    ap.add_argument("--focus", default="vector-search")
    ap.add_argument("--query", default="notes")
    ap.add_argument("--max-files", type=int, default=60)
    ap.add_argument("--max-bytes", type=int, default=18000)
    ap.add_argument("--out", default="reports/glm51")
    args = ap.parse_args()

    repo = pathlib.Path(args.repo).expanduser().resolve()
    if not repo.exists():
        sys.exit(f"Repo not found: {repo}")
    out_dir = repo / args.out
    out_dir.mkdir(parents=True, exist_ok=True)

    git_status = run(["git", "status", "--short"], repo)
    git_diff = run(["git", "diff", "--", "."], repo)
    context, total, files = collect_files(repo, args.max_files, args.max_bytes)

    results = []
    for phase_name, phase_goal in CASCADE:
        prompt = build_prompt(repo, args.focus, args.query, context, files, git_status, git_diff, phase_name,
                               phase_goal)
        print(f"Running GLM-5.1 phase: {phase_name}", file=sys.stderr)
        result = call_openrouter(prompt)
        result["phase"] = phase_name
        results.append(result)
        time.sleep(1)

    stamp = time.strftime("%Y%m%d-%H%M%S")
    json_path = out_dir / f"glm51-review-{stamp}.json"
    md_path = out_dir / f"glm51-review-{stamp}.md"

```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>


```

json_path.write_text(json.dumps(results, indent=2), encoding="utf-8")

with md_path.open("w", encoding="utf-8") as f:
    f.write(f"# GLM-5.1 Review: {args.focus} / query={args.query!r}\n\n")
    f.write(f"Repo: `{repo}`\n- Model: `{MODEL}`\n- Files sampled: {len(files)}\n- Context chars: {total}\n\n")
    for item in results:
        f.write(f"## Phase: {item['phase']}\n\n")
        f.write(item["content"].strip() + "\n\n")
        if item.get("reasoning_details"):
            f.write("<details><summary>Reasoning details metadata</summary>\n\n")
            f.write("`json\n" + json.dumps(item["reasoning_details"], indent=2)[:8000] +
                "\n`</details>\n\n")
            f.write(f"Usage: `{item.get('usage')}`\n\n")
    print(f"Wrote {md_path}\nWrote {json_path}")

if __name__ == "__main__":
    main()

```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Test commands

Run these commands from your Mac. They assume the two scripts above are saved in `~/dev/obsidian_rag` and `~/dev/obsidian_rag/tools/`.

```
# One-time setup
export OPENROUTER_API_KEY="sk-or-..."
cd ~/dev/obsidian_rag
bash ./setup_glm51_openrouter_review.sh

# Optional OpenClaw credential storage, if your installed version supports it
openclaw auth set openrouter:default --key "$OPENROUTER_API_KEY"
openclaw gateway run

# Run the focused review
source .venv-glm51-review/bin/activate
python tools/glm51_review.py --repo . --focus vector-search --query notes

# Repeat with known weak queries
python tools/glm51_review.py --repo . --focus vector-search --query note
python tools/glm51_review.py --repo . --focus vector-search --query "what am I working on"
python tools/glm51_review.py --repo . --focus vector-search --query "latest facts"

# Run local regression tests after applying fixes suggested by the report
pytest -q
pytest -q tests -k "retrieval or vector or search or notes"
python -m compileall .
git diff --check
```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>

Three-Mac command variants

Use this section for your actual deployment. The default path is: sync canmore's repo to lobster, run GLM-5.1 review on lobster, and optionally trigger or retrieve reports from macbook over Tailscale.

```
# Recommended: run these on lobster, the OpenClaw host.
# Option A: clone canmore's canonical repo over LAN/SSH
mkdir -p ~/dev
cd ~/dev
git clone canmore:~/dev/obsidian_rag obsidian_rag
cd ~/dev/obsidian_rag

# Option B: if lobster already has a clone, sync it from canmore
cd ~/dev/obsidian_rag
git remote -v
git fetch --all --prune
git pull --ff-only

# Option C: if Git-over-SSH is not configured, use rsync from canmore to lobster
rsync -az --delete \
  --exclude .git \
  --exclude .venv \
  --exclude .venv-glm51-review \
  canmore:~/dev/obsidian_rag/ ~/dev/obsidian_rag/

# Run the review on lobster, against lobster's local working copy
export OPENROUTER_API_KEY="sk-or-..."
cd ~/dev/obsidian_rag
./setup_glm51_openrouter_review.sh
source .venv-glm51-review/bin/activate
python tools/glm51_review.py --repo . --focus vector-search --query notes

# From macbook, trigger lobster over Tailscale without moving the canonical role
ssh lobster 'cd ~/dev/obsidian_rag && git pull --ff-only && \
  source .venv-glm51-review/bin/activate && \
  python tools/glm51_review.py --repo . --focus vector-search --query notes'

# Bring reports back to macbook if desired
rsync -az lobster:~/dev/obsidian_rag/reports/glm51/ \
  ~/dev/obsidian_rag/reports/glm51/
```

Review report workflow

- Open reports/glm51/glm51-review-*.md after each run and sort findings into quick fixes, tests, and architectural follow-ups.
- Apply one refactor at a time, then rerun the same --query notes review to check whether GLM-5.1 still finds the same sensitivity failure.
- Keep a stable regression test for every failure mode that GLM-5.1 identifies; once a bug is fixed, the test becomes the guardrail.
- If the model returns too much advice, lower --max-files, add a narrower --focus, or temporarily add only the retrieval files to a branch.

Recommended first run

```
cd ~/dev/obsidian_rag
export OPENROUTER_API_KEY="sk-or-..."
chmod +x setup_glm51_openrouter_review.sh tools/glm51_review.py
./setup_glm51_openrouter_review.sh
source .venv-glm51-review/bin/activate
python tools/glm51_review.py --repo . --focus vector-search --query notes
```

1. OpenRouter chat completions endpoint: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
2. OpenRouter GLM 5.1 model page: <https://openrouter.ai/z-ai/glm-5.1>
3. OpenRouter OpenClaw integration guide: <https://openrouter.ai/docs/guides/coding-agents/openclaw-integration>
4. OpenRouter reasoning tokens guide: <https://openrouter.ai/docs/use-cases/reasoning-tokens>